



École Polytechnique de Montréal

Département de génie informatique et génie logiciel

INF8901 — Projet de recherche personnelle

**Rapport d'avancement partiel**

Travail présenté à Benoit Ozell

Par:  
Félix Gagnon, 2209061

February 28, 2026

# Contents

|          |                                                                                 |           |
|----------|---------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                             | <b>1</b>  |
| <b>2</b> | <b>Littérature</b>                                                              | <b>1</b>  |
| 2.1      | Représentation d'une scène . . . . .                                            | 1         |
| 2.1.1    | Diversité des pipelines de rendu . . . . .                                      | 2         |
| 2.1.2    | Formats d'encodage des éléments de scène . . . . .                              | 2         |
| 2.1.3    | Le défi de la fragmentation . . . . .                                           | 2         |
| 2.2      | Cheveux . . . . .                                                               | 2         |
| 2.2.1    | État de l'art du rendu des cheveux . . . . .                                    | 2         |
| 2.2.2    | Comparaison des modèles de rendu des cheveux . . . . .                          | 4         |
| 2.3      | Processus gaussiens . . . . .                                                   | 4         |
| 2.3.1    | Noyaux . . . . .                                                                | 5         |
| 2.3.2    | Conditionnement . . . . .                                                       | 6         |
| 2.3.3    | Échantillonnage . . . . .                                                       | 6         |
| 2.4      | Surfaces implicites . . . . .                                                   | 6         |
| 2.4.1    | Surfaces implicites par processus gaussiens (GPIS) . . . . .                    | 6         |
| 2.4.2    | Représentation d'élément par processus gaussiens . . . . .                      | 7         |
| 2.4.3    | Dérivées des processus gaussiens et normales de surface . . . . .               | 7         |
| 2.5      | Une théorie unifiée du transport lumineux avec géométrie stochastique . . . . . | 8         |
| 2.5.1    | Algorithme . . . . .                                                            | 8         |
| <b>3</b> | <b>Implementation</b>                                                           | <b>8</b>  |
| 3.1      | Choix du moteur de rendu . . . . .                                              | 8         |
| 3.2      | Mitsuba 3 . . . . .                                                             | 9         |
| 3.2.1    | L'API . . . . .                                                                 | 9         |
| 3.3      | Preuve de concept . . . . .                                                     | 10        |
| 3.4      | Difficultés de développement . . . . .                                          | 10        |
| 3.4.1    | Configuration de l'environnement de compilation . . . . .                       | 10        |
| 3.4.2    | Développement de la bibliothèque de processus gaussien . . . . .                | 10        |
| 3.5      | Résultats et analyse . . . . .                                                  | 12        |
| 3.6      | Étapes suivantes . . . . .                                                      | 12        |
| <b>4</b> | <b>Evaluation</b>                                                               | <b>12</b> |
| 4.1      | Méthodologie . . . . .                                                          | 12        |
| 4.1.1    | Évaluation de la qualité d'image . . . . .                                      | 12        |
| 4.1.2    | Performances de rendu . . . . .                                                 | 13        |
| <b>5</b> | <b>Discussion</b>                                                               | <b>13</b> |

# 1 Introduction

Les algorithmes de tracé de rayon sont omniprésents dans les moteurs de rendu de pointe. Les traceurs de chemin de Monte Carlo ont été utilisés pour rendre des maillages, tandis que les volumes sont rendus à l’aide du tracé de rayon volumétrique. Certains médias se situent entre les représentations volumétriques et surfaciques, comme les cheveux et la fourrure.

Un nouveau paradigme prometteur a récemment été présenté par Seyb et al. dans “From Microfacets to Participating Media: A Unified Theory of Light Transport with Stochastic Geometry” [1]. Ils proposent un nouvel algorithme de tracé de rayon pour faire le rendu de surfaces implicites définies par processus de Gauss (GPIS), ouvrant ainsi la voie à un continuum d’éléments pouvant être représentés avec le même algorithme.

Cette recherche vise à déterminer si le rendu des cheveux peut être réalisé efficacement à l’aide de ce nouveau continuum. Ce rapport d’avancement partiel présente les résultats et l’orientation du projet de recherche, ainsi que les progrès réalisés sur la preuve de concept concernant un plugin de forme GPIS pour Mitsuba 3.

La revue de littérature présente la théorie importante liée à la difficulté d’utiliser plusieurs algorithmes de rendu pour une même scène, ainsi que l’état de l’art sur le rendu des cheveux et de la fourrure. Elle introduit également les notions essentielles de processus gaussiens et de GPIS.

Les GPIS ne sont pas nouveaux, mais leur utilisation pour le rendu de scènes est relativement récente et n’a été démontrée jusqu’à présent que par l’équipe de Seyb et al. [1]. Il serait pertinent d’implémenter l’algorithme présenté dans le moteur de rendu largement utilisé Mitsuba 3. Dans ce rapport, je présente le processus de décision ayant mené au choix du moteur de rendu et l’état actuel de la preuve de concept.

Enfin, je décris comment l’implémentation GPIS sera comparée à l’approche de référence basée sur le tracé de rayon Monte Carlo.

## 2 Littérature

### 2.1 Représentation d’une scène

La représentation de scène en rendu fait référence à la manière dont les environnements 3D et leurs éléments constitutifs sont organisés, stockés et décrits pour être utilisés dans les pipelines de rendu. Les formats modernes de description de scène, tels que Universal Scene Description (USD) et glTF (GL Transmission Format), constituent la base de cette organisation.

USD, développé par Pixar, est conçu pour l’interopérabilité entre différents logiciels 3D, en mettant l’accent sur des descriptions de scène éditables destinées aux outils de création numérique. De son côté, glTF est optimisé pour la diffusion de contenu et le rendu final, notamment pour le web et les applications en temps réel. Ces formats encodent différents types d’éléments, notamment des maillages d’objet, des personnages dotés de systèmes squelettiques et de cheveux, ainsi que des médias participatifs comme la fumée, le brouillard ou la vapeur.

### 2.1.1 Diversité des pipelines de rendu

Différents types d'éléments de la scène nécessitent des approches de rendu spécialisées. Par exemple, Blender Cycles utilise un tracé de rayon volumétrique pour rendre les milieux participatifs, tout en employant le lancer de rayons standard pour les maillages d'objets. Cette séparation reflète les différences fondamentales entre l'interaction de la lumière avec les surfaces et avec les volumes.

### 2.1.2 Formats d'encodage des éléments de scène

Le paysage des formats d'encodage des éléments est très fragmenté, chaque type d'élément nécessitant sa propre représentation spécialisée :

**Maillages** Les maillages de géométrie utilisent généralement des approches similaires au format Wavefront OBJ, qui encode les positions des sommets, les normales, les coordonnées de texture ainsi que les indices de sommets définissant la topologie des faces triangulaires ou polygonales.

**Milieus participants (Participating Media)** Les données volumétriques telles que la fumée, le brouillard, le feu ou les nuages sont couramment stockées à l'aide de formats comme OpenVDB, une bibliothèque open source initialement développée par DreamWorks Animation pour manipuler des données volumétriques creuses (sparse). OpenVDB permet un stockage efficace de structures voxelisées pour la fumée, le feu et les simulations de fluides grâce à une structure hiérarchique clairsemée qui ne conserve que les régions occupées, réduisant drastiquement les besoins mémoire comparativement aux grilles volumétriques denses.

**Cheveux et fourrure basés sur des brins (Strand-Based Hair and Fur)** Les cheveux et la fourrure posent des défis d'encodage particuliers. Les mèches sont généralement représentées sous forme de courbes, souvent à l'aide de splines cubiques de Catmull-Rom ou de courbes de Bézier, définies par des points de contrôle décrivant la trajectoire du brin. Ces représentations incluent des informations de largeur par sommet le long de la spline afin de capturer l'amincissement progressif du cheveu de la racine vers la pointe. L'encodage doit donc stocker non seulement la courbe centrale, mais également les paramètres de largeur permettant au moteur de rendu de reconstruire correctement la nature volumétrique du cheveu lors du rendu.

### 2.1.3 Le défi de la fragmentation

Cette diversité de formats de représentation implique que les pipelines de rendu doivent prendre en charge plusieurs types d'éléments, chacun possédant des structures mémoire, des routines d'intersection et des modèles d'ombrage distincts.

Une approche unifiée capable de traiter surfaces et volumes avec une représentation unique pourrait simplifier considérablement les architectures de rendu. Cela constitue une partie de la motivation derrière l'exploration de techniques telles que les GPIS.

## 2.2 Cheveux

### 2.2.1 État de l'art du rendu des cheveux

Les cheveux et la fourrure sont omniprésents dans les médias rendus, des jeux vidéo aux personnages de films d'animation. Encore aujourd'hui, le rendu réaliste des fibres demeure un défi, tant en raison de leur complexité géométrique que de la diffusion lumineuse

complexe d'une fibre à l'autre. La plupart des techniques modernes trouvent leur origine soit dans l'approche des *hair cards* proposée par Kajiya et Kay [2], soit dans le paradigme de diffusion par fibres introduit par Marschner et al. [3]. Récemment, la recherche s'est concentrée sur la formulation de modèles de diffusion compatibles avec les traceurs de chemin Monte Carlo; par exemple, Chiang et al. [4] ont développé un modèle pratique pour les artistes, désormais largement adopté comme modèle de shading des cheveux dans des outils tels que Houdini et Mitsuba 3.

**Modèle de Kajiya et Kay** Le modèle de Kajiya et Kay (1989) [2] repose sur une représentation polygonale des cheveux, ce qui lui permet de s'intégrer facilement aux pipelines de rendu rasterisés. L'approche modélise les cheveux comme des fibres cylindriques et utilise un modèle d'ombrage anisotrope prenant en compte la direction tangente du brin.

Le modèle capture la caractéristique la plus visible de la diffusion par fibres : l'apparition d'un reflet linéaire perpendiculaire à la direction des fibres dans l'image. Cette observation repose sur le fait que la réflexion d'un faisceau parallèle sur la surface d'un cylindre produit un cône centré sur l'axe du cheveu. Le modèle combine un terme diffus (proportionnel au cosinus de l'angle d'incidence) avec un reflet spéculaire centré sur ce cône.

Cependant, le modèle de Kajiya et Kay présente des limites importantes : il traite les cheveux comme des cylindres opaques et ne tient pas compte de la transmission ni des réflexions internes. Or, le cheveu étant un matériau diélectrique et parfois translucide, cette omission conduit à un rendu moins réaliste, incapable de reproduire correctement les reflets colorés et les effets de rétroéclairage.

**Modèle de Marschner** Le modèle de Marschner (2003) [3] constitue une avancée majeure par rapport à Kajiya et Kay en proposant une modélisation plus réaliste des fibres. Le modèle considère les cheveux comme des cylindres diélectriques rugueux, permettant de simuler correctement la réflexion interne et la transmission.

Marschner identifie trois modes principaux d'interaction de la lumière avec un cheveu :

- **R (Reflection)** : la lumière se réfléchit sur la surface externe de la fibre, produisant le reflet blanc principal.
- **TT (Transmission-Transmission)** : la lumière pénètre dans la fibre, traverse son intérieur et ressort de l'autre côté, produisant une diffusion vers l'avant visible en contre-jour.
- **TRT (Transmission-Reflection-Transmission)** : la lumière pénètre dans la fibre, se réfléchit sur la surface interne opposée, puis ressort, générant le reflet secondaire coloré.

La fonction de diffusion peut s'écrire :

$$\begin{aligned}
S(\theta_i, \theta_r, \phi_i, \phi_r) = & \frac{M_R(\theta_h) N_R(\eta'(\eta, \theta_d), \phi)}{\cos^2(\theta_d)} \\
& + \frac{M_{TT}(\theta_h) N_{TT}(\eta'(\eta, \theta_d), \phi)}{\cos^2(\theta_d)} \\
& + \frac{M_{TRT}(\theta_h) N_{TRT}(\eta^*(\phi_h, \theta_d), \phi)}{\cos^2(\theta_d)}
\end{aligned} \tag{1}$$

où  $M$  représente la fonction de diffusion longitudinale (modélisée par des gaussiennes) et  $N$  la fonction de diffusion azimutale. Le modèle utilise des coordonnées sphériques avec  $\theta$  pour l'angle longitudinal et  $\phi$  pour l'angle azimutal. Plusieurs angles dérivés sont employés :

- L'angle de différence :  $\theta_d = (\theta_r - \theta_i)/2$
- L'azimut relatif :  $\phi = \phi_r - \phi_i$
- Les demi-angles :  $\theta_h = (\theta_i + \theta_r)/2$  et  $\phi_h = (\phi_i + \phi_r)/2$

En principe, ce schéma peut être prolongé pour des transmissions avec  $n$  réflexions internes (TRRT, TRRRT, etc.), mais l'impact visuel des termes d'ordre supérieur au TRT est généralement négligeable.

### 2.2.2 Comparaison des modèles de rendu des cheveux

La figure 1 illustre, à gauche, des cheveux rendus avec le modèle de Kajiya-Kay, au centre avec le modèle de Marschner, et à droite des cheveux réels sous des conditions d'éclairage similaires. On observe clairement que le modèle de Marschner permet une meilleure reproduction des reflets secondaires colorés et offre un aspect moins plat que le modèle de Kajiya-Kay, bien qu'il conserve une certaine uniformité et ne capture pas encore parfaitement la richesse visuelle des cheveux réels.

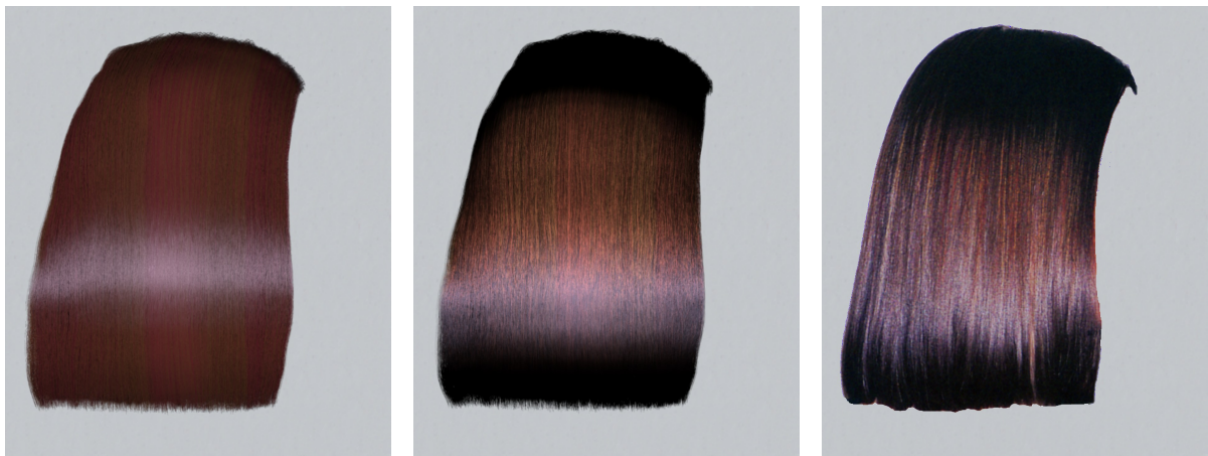


Figure 1: Comparaison des modèles de rendu des cheveux : Kajiya-Kay (gauche), Marschner (centre) et cheveux réels (droite) sous un éclairage similaire. Image tirée de Marschner et al. [3].

## 2.3 Processus gaussiens

Les processus gaussiens (GP) constituent un cadre d'apprentissage automatique qui considère l'ensemble des fonctions possibles pouvant approximer un jeu de données, en assignant une probabilité à chacune d'elles [5]. L'hypothèse clé est que des points d'entrée similaires doivent produire des sorties similaires, ce qui permet au modèle de fournir à la fois une estimation ponctuelle et un degré d'incertitude pour chaque prédiction.

Les GP sont définis par deux fonctions :

1. **La fonction moyenne**  $\mu(\mathbf{x})$ , qui représente les croyances a priori sur la fonction sous-jacente
2. **Le noyau de covariance**  $k(\mathbf{x}, \mathbf{x}')$ , qui représente la corrélation entre deux points

Les GP fournissent une moyenne postérieure représentant la valeur prédite ainsi qu'une variance exprimant l'incertitude associée.

### 2.3.1 Noyaux

La fonction noyau est utilisée pour définir la matrice de covariance en encodant la corrélation entre deux points. Le choix du noyau est crucial en modélisation par GP, car il incorpore des hypothèses sur la régularité, la périodicité et d'autres propriétés de la fonction sous-jacente [6].

**Noyau exponentiel quadratique (RBF)** Le noyau exponentiel quadratique (ou noyau RBF) est défini par :

$$k_{SE}(\mathbf{x}, \mathbf{x}') = \sigma^2 \exp \left( -\frac{1}{2\ell^2} \|\mathbf{x} - \mathbf{x}'\|^2 \right) \quad (2)$$

où  $\sigma^2$  est la variance du signal et  $\ell$  le paramètre d'échelle.

Ce noyau est particulièrement pertinent pour les GPIS, car il produit des surfaces infiniment différentiables et donc très lisses. Le paramètre d'échelle  $\ell$  offre un contrôle intuitif sur les caractéristiques de la surface : des valeurs faibles produisent des variations rapides entre les points, tandis que des valeurs plus grandes imposent des transitions plus douces. Cette interprétabilité et cette flexibilité en font un choix adapté à la représentation d'objets naturels aux frontières continues et régulières.

Les équations des noyaux sont tirées de Martens et al. [6].

**Noyau Thin Plate Spline** Dans le cas tridimensionnel, le noyau *thin plate spline* est donné par :

$$k_{TP}(\mathbf{x}, \mathbf{x}') = 2d^3 - 3Rd^2 + R^3 \quad (3)$$

où :

- $d = \|\mathbf{x} - \mathbf{x}'\|^2$ ,
- $R$  est un hyperparamètre définissant le rayon de support.

Ce noyau est motivé par la minimisation de l'énergie de flexion, analogue à la déformation d'une plaque mince. Pour les GPIS, cette propriété est avantageuse, car elle génère naturellement des surfaces à courbure minimale, produisant des interpolations lisses entre des points de données clairsemés. Toutefois, contrairement au noyau exponentiel quadratique, son efficacité est limitée à un domaine fini défini par  $R$ , et il offre moins de flexibilité pour ajuster les propriétés de la surface.

### 2.3.2 Conditionnement

Le conditionnement d'un processus gaussien consiste à incorporer des données observées afin d'affiner les prédictions. Les équations de conditionnement et d'échantillonnage sont dérivées dans Martens et al. [6] et Seyb et al. [1]; nous adoptons la notation de Seyb et al. [1] qui est plus explicite. Étant donné des mesures  $\mathbf{m}$  aux positions  $\mathbf{C}$ , le processus gaussien conditionné s'écrit :

$$f \sim \text{GP}(\mu_{|\zeta_m}(\mathbf{x}), k_{|\zeta_m}(\mathbf{x}, \mathbf{y})) \quad (4)$$

avec :

$$\mu_{|\zeta_m}(\mathbf{x}) = \mu(\mathbf{x}) + k(\mathbf{x}, \mathbf{C})k(\mathbf{C}, \mathbf{C})^{-1}(\mathbf{m} - \mu(\mathbf{C})) \quad (5)$$

$$k_{|\zeta_m}(\mathbf{x}, \mathbf{y}) = k(\mathbf{x}, \mathbf{y}) - k(\mathbf{x}, \mathbf{C})k(\mathbf{C}, \mathbf{C})^{-1}k(\mathbf{C}, \mathbf{y}) \quad (6)$$

### 2.3.3 Échantillonnage

L'échantillonnage d'un processus gaussien peut être réalisé via :

$$f(\mathbf{X}) = \mu(\mathbf{X}) + k(\mathbf{X}, \mathbf{X})^{1/2}\boldsymbol{\eta} \quad (7)$$

où :

- $\boldsymbol{\eta} \sim \mathcal{N}(0, 1)$
- $\mathbf{A}^{1/2}$  est la racine carrée matricielle telle que  $\mathbf{A} = \mathbf{A}^{1/2}\mathbf{A}^{1/2}$

## 2.4 Surfaces implicites

Une surface implicite est définie par une fonction  $f(X)$ , où la surface correspond à l'ensemble des points tels que  $f = 0$ . Par convention, les points où  $f > 0$  sont considérés comme étant à l'intérieur d'un objet et ceux où  $f < 0$  à l'extérieur, bien que cette convention puisse être inversée.

### 2.4.1 Surfaces implicites par processus gaussiens (GPIS)

Les surfaces implicites définies par processus gaussiens (GPIS) ont été introduites par Williams and Fitzgibbon [7] comme une approche probabiliste de reconstruction de surface [6]. La moyenne du processus gaussien décrit une variété représentant l'élément géométrique.

Les données d'entraînement d'un GPIS comprennent :

- Un ensemble de points sur la surface (où  $f = 0$ )
- Un ensemble de points connus à l'intérieur de la surface (où  $f > 0$ )
- Un ensemble de points connus à l'extérieur de la surface (où  $f < 0$ )

Si des normales associées aux points sont disponibles, on peut générer automatiquement des points intérieur/extérieur en décalant légèrement le long et contre la normale.

Le processus gaussien apprend une fonction qui interpole de manière lisse toutes les observations. La moyenne du GP représente la fonction implicite la plus probable, tandis que la variance fournit des estimations d'incertitude. La surface implicite correspond à l'ensemble des points où  $f(X) = 0$ .



### 2.4.2 Représentation d'élément par processus gaussiens

Les GPIS sont particulièrement intéressants pour représenter des formes numérisées à partir de données LiDAR ou de caméras RGB-D. À partir d'un nuage de points, il est possible de conditionner le processus gaussien en définissant les points du nuage comme observations (avec  $f = 0$ ). Cela crée une variété qui peut être rendue à l'aide d'algorithmes de *ray marching* par recherche de racines, afin de déterminer l'intersection avec la variété définie par  $f(X) = 0$ .

Pour les assets basés sur des maillages, il est possible de conditionner le GP en :

1. Ajoutant chaque sommet du maillage comme observation de surface ( $f = 0$ )
2. Exploitant les normales aux sommets pour inclure des points à l'extérieur du modèle ( $f = -1$ ) et à l'intérieur ( $f = +1$ ) en décalant dans la direction de la normale

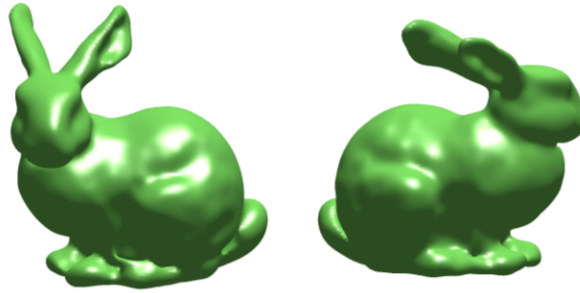


Figure 2: Le Stanford bunny défini par 800 points de surface, un point intérieur  $+1$ , et une sphère de 80 points extérieurs  $-1$  générés par l'algorithme marching cubes. Image tirée de Williams and Fitzgibbon [7].

Pour les nuages de points issus de scans LiDAR, les normales ainsi que les points intérieurs et extérieurs doivent être déduits de la géométrie locale. Cela peut être réalisé en estimant les normales de surface à partir du gradient du processus gaussien. [6]

Le GP conditionné définit une surface implicite qui interpole de manière lisse les données observées tout en fournissant des estimations d'incertitude loin des observations. Cette représentation probabiliste est particulièrement utile pour traiter des données capteurs bruitées et des observations clairsemées.

### 2.4.3 Dérivées des processus gaussiens et normales de surface

Une propriété fondamentale des processus gaussiens est que les dérivées d'un GP sont également des processus gaussiens. Cette propriété est essentielle pour les GPIS, car les normales de surface sont dérivées du gradient de la fonction implicite.

**Processus gaussiens dérivés** En raison de la linéarité de l'opérateur de dérivation, la dérivée d'un processus gaussien prend la forme :

$$GP'(\mu(\mathbf{x}), k(\mathbf{x}, \mathbf{y})) = GP(\mu'(\mathbf{x}), k_{xy}(\mathbf{x}, \mathbf{y})) \quad (8)$$

où :

$$k_{xy}(\mathbf{x}, \mathbf{y}) = \frac{\partial^2 k(\mathbf{x}, \mathbf{y})}{\partial \mathbf{x} \partial \mathbf{y}} \quad (9)$$

Cela signifie que la covariance entre les dérivées peut être calculée en prenant la dérivée croisée du second ordre du noyau original.

**Calcul des normales sur les GPIS** Pour une surface implicite définie par  $f(X) = 0$ , la normale de surface en tout point est donnée par le gradient normalisé :

$$\mathbf{n}(\mathbf{x}) = \frac{\nabla f(\mathbf{x})}{\|\nabla f(\mathbf{x})\|} \quad (10)$$

## 2.5 Une théorie unifiée du transport lumineux avec géométrie stochastique

Seyb et al. [1] ont utilisé les GPIS pour définir un modèle contrôlable capable de représenter tous les éléments d’une scène comme un continuum. Cela permet la représentation d’une scène à l’aide d’un modèle de transport lumineux unique.

### 2.5.1 Algorithme

L’algorithme de tracé de rayon pour les GPIS s’appuie sur la fonction `nextHit` pour trouver les intersections de rayons. Cette fonction détermine la première distance  $t$  pour laquelle le rayon  $\mathbf{x} + t\mathbf{w}$  intersecte la surface, définie par  $f(\mathbf{x}_t) = 0$ . Pour trouver l’intersection, on génère une réalisation 1D du processus gaussien le long du rayon et on localise le changement de signe de cette fonction 1D. On échantillonne le processus gaussien en  $n$  points le long du rayon, puis on interpole entre ces points pour obtenir une réalisation 1D continue adaptée à la recherche de racines. Une fois le point d’intersection obtenu, on calcule la normale par la méthode du gradient et, avec le point d’intersection et la normale, on peut évaluer le BSDF exactement comme dans un tracé de rayon standard.

## 3 Implementation

### 3.1 Choix du moteur de rendu

Comme expliqué précédemment, pour rendre une GPIS via le transport de la lumière, il faut pouvoir échantillonner les intersections de rayons à partir d’un processus gaussien conditionné et retourner des interactions de surface comme dans un algorithme classique de transport de la lumière.

Une implémentation complète depuis zéro pourrait être réalisée en utilisant CUDA ou des compute shaders OpenGL. Cela impliquerait de définir la traversal de scène, les routines d’intersection de rayons, l’échantillonnage d’importance multiple, les interactions de surface, ainsi que le parsing de scène et le chargement en mémoire GPU depuis le CPU. Cette approche donne un contrôle total sur le moteur mais impose d’implémenter beaucoup de code d’infrastructure de lancer de rayons sans rapport direct avec les GPIS.

En alternative, on pourrait créer un noeud personnalisé pour un moteur contemporain comme Blender Cycles [8] ou Autodesk Arnold [9]. Ces moteurs utilisent le tracé de rayon Monte Carlo pour les médias volumétriques et les objets maillés, et permettent la création de plugins ou `nodes` pour définir des comportements personnalisés. Malheureusement, ils ne proposent pas suffisamment de flexibilité pour définir des routines d’intersection de

rayons personnalisées basées sur l'échantillonnage d'un processus gaussien, fonctionnalité essentielle pour intégrer les GPIS.

Une troisième option consiste à utiliser des moteurs orientés recherche. Tungsten [10] est le moteur employé par Seyb et al. [1] and Xu et al. [11]. Bien que l'implémentation GPIS y soit disponible, le projet public n'est pas conçu pour un usage large et manque de documentation. La version publiée ne supporte pas le chargement des fichiers de cheveux ni la conditionnement des processus gaussiens à partir des splines récupérées, ce qui exigerait une rétro-ingénierie du code.

Mitsuba 3 [12] constitue une option populaire pour la recherche en rendu. Il s'agit d'un moteur retargetable développé à l'EPFL qui expose une API Python pour définir des plugins personnalisés tels que des intégrateurs ou des BSDF. Bien que l'API Python ne permette pas de définir des routines d'intersection personnalisées, cela reste possible en implémentant une forme personnalisée dans le code C++ [17].

Étant un projet open source orienté recherche, Mitsuba 3 représente un bon compromis : par rapport à une implémentation complète à partir de zéro, il permet de gagner du temps en réutilisant les routines de lancer de rayons existantes. Il est activement maintenu et documenté, contrairement à Tungsten, et offre une flexibilité significative tout en supportant l'exécution retargetable, contrairement à Cycles et Arnold.

## 3.2 Mitsuba 3

D'après la page GitHub :

Mitsuba 3 est un système de rendu orienté recherche pour la simulation du transport de la lumière direct et inverse développé à l'EPFL en Suisse. Il comprend une bibliothèque cœur et un ensemble de plugins implémentant des fonctionnalités allant des matériaux et sources lumineuses jusqu'à des algorithmes de rendu complets. Mitsuba 3 est retargetable : cela signifie que les implémentations et structures de données sous-jacentes peuvent être transformées pour accomplir différentes tâches. [12]

### 3.2.1 L'API

Les éléments suivants sont tirés de la documentation officielle de Mitsuba 3 [12].

**Plugins** L'architecture de tracé de rayon de Mitsuba est séparée en composants appelés plugins, qui représentent les systèmes nécessaires au rendu. Les composants pertinents pour l'implémentation d'une forme basée GPIS seront présentés ci-après.

**Intégrateur** Les intégrateurs définissent la stratégie de résolution de l'équation du transport de la lumière. Ils gèrent la traversal de scène et contrôlent des paramètres tels que la profondeur maximale de rebonds. L'intégrateur de tracé de rayon est un plugin existant qui prend en charge l'essentiel du calcul et des optimisations utiles comme l'échantillonnage d'importance multiple.

**Shapes** Les shapes définissent les surfaces marquant l'intersection entre différents types de matériaux et peuvent aussi définir des intersections avec des volumes. Toutes les shapes doivent fournir une boîte englobante utilisée par l'intégrateur.

Les shapes existantes incluent des primitives basiques (cubes, sphères, plans) ainsi que des shapes maillées chargeant des fichiers OBJ ou PLY.

Ce projet nécessitera la création d'une shape personnalisée nommée `gpis_shape` capable de charger des sommets et de conditionner un processus gaussien. L'API Shape est responsable d'implémenter la routine d'intersection de rayons utilisée par l'intégrateur et qui retourne un objet d'interaction de surface.

**BSDF** Les BSDFs (Bidirectional Scattering Distribution Functions) représentent le modèle de dispersion de la surface. L'implémentation d'un BSDF personnalisé ne semble pas nécessaire, car la diffusion d'une GPIS doit se comporter comme un équivalent maillé. Une BSDF cheveux existante sera réutilisée (modèle de Chiang et al. [4]).

**Sampler** Les samplers génèrent les nombres aléatoires pour l'intégration Monte Carlo et contrôlent la qualité des échantillons.

### 3.3 Preuve de concept

Une preuve de concept a été réalisée en ajoutant un plugin `gpis_sphere` à Mitsuba 3. La forme conditionne un processus gaussien à partir de 201 observations : 100 points de surface, 100 points extérieurs et un point intérieur. Le renderer représente l'intersection par la moyenne du processus gaussien conditionné. La sphère est placée dans une scène de type Cornell Box via la description de scène XML de Mitsuba, les murs étant des rectangles existants et l'éclairage fourni par une source de type area light.

### 3.4 Difficultés de développement

#### 3.4.1 Configuration de l'environnement de compilation

La mise en oeuvre de la preuve de concept a présenté plusieurs défis techniques. La compilation de Mitsuba 3 a nécessité une configuration précise des dépendances. Plutôt que de travailler directement sur un fork de Mitsuba, j'ai configuré le système pour lier Mitsuba 3 comme bibliothèque externe au plugin via CMake, ce qui a exigé une compréhension de la pipeline de compilation de Mitsuba 3.

#### 3.4.2 Développement de la bibliothèque de processus gaussien

L'écriture d'un utilitaire pour les processus gaussiens a demandé plusieurs itérations. J'ai d'abord tenté d'utiliser `libgp`, mais son interface n'exposait pas la fonctionnalité nécessaire pour l'échantillonnage. Ensuite, une implémentation basée sur Eigen3 a posé des conflits de types avec DrJit.

DrJit est le compilateur JIT utilisé par Mitsuba 3, permettant la compilation vers CUDA, LLVM ou un backend scalaire à partir d'un code unifié. Le traitement SIMD de la routine d'intersection a créé des conflits avec les types standards C++ et Eigen3; la solution a été d'implémenter la classe de processus gaussien entièrement en syntaxe DrJit pour garantir la compatibilité.

Actuellement, `gaussian_process.h` est une classe simple permettant de conditionner un modèle à partir d'observations de surface, extérieures et intérieures en utilisant un noyau RBF. Pour cette preuve de concept, l'échantillonnage corrélé à partir du postérieur

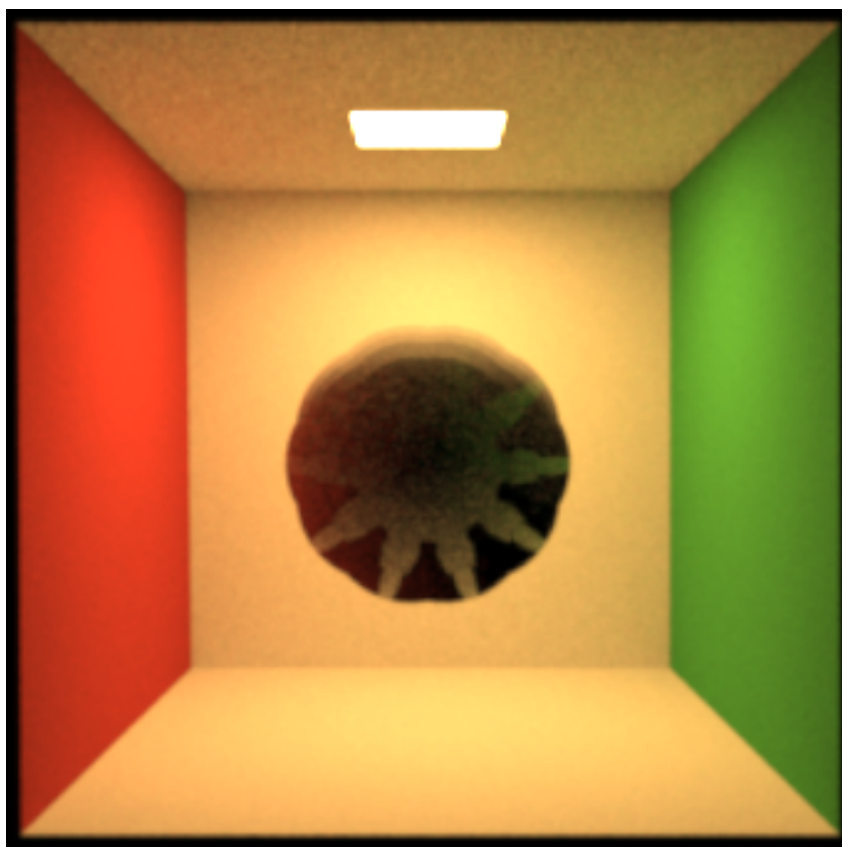


Figure 3: Scène Cornell Box avec une sphère GPIS rendue en 256x256 utilisant 128 échantillons par pixel. Temps de rendu : 30 minutes.

n’a pas encore été implémenté : la routine d’intersection utilise un simple ray marching pour trouver les intersections avec la moyenne du processus.

### 3.5 Résultats et analyse

La figure 3 montre une Cornell Box avec une sphère GPIS au centre. La surface présente des bosselures aux points conditionnés, attendu avec seulement 100 points de surface et un noyau RBF.

Le comportement lumineux confirme l’intégration correcte avec le système BSDF de Mitsuba 3 : un fort reflet apparaît au sommet de la sphère et les côtés reflètent les couleurs des murs, validant la compatibilité avec les BSDF existants.

Une approximation de l’implémentation actuelle est l’utilisation de normales de sphère régulières plutôt que la méthode basée sur le gradient, d’où des artefacts triangulaires noirs. La sphère montre également une apparence partiellement translucide au sommet, une conséquence souhaitable de l’approche GPIS.

### 3.6 Étapes suivantes

La prochaine phase se concentrera sur trois améliorations clés : premièrement, implémenter l’échantillonnage corrélé depuis la distribution postérieure; deuxièmement, ajouter le calcul des normales basé sur le gradient. Avec ces deux améliorations, la sphère sera rendue conformément à l’approche décrite par Seyb et al. [1].

Ensuite, le système sera étendu pour rendre des maillages en conditionnant le processus sur les sommets importés depuis des fichiers OBJ. L’augmentation du nombre de points d’observation ralentira le rendu, rendant pertinente l’approche de matrice creuse proposée par Xu et al. [11] pour préserver les performances.

Enfin, j’étudierai quel noyau convient le mieux pour le rendu des cheveux afin d’éviter la fusion indésirable des mèches, un problème encore largement inexploré dans la littérature actuelle sur GPIS et le rendu des cheveux.

## 4 Evaluation

### 4.1 Méthodologie

L’évaluation de l’implémentation GPIS se fera selon deux métriques clés : la qualité d’image et les performances de rendu.

#### 4.1.1 Évaluation de la qualité d’image

Nous utiliserons l’erreur quadratique moyenne (MSE) pour comparer des images rendues avec un tracé de rayon standard à celles rendues avec l’objet GPIS. La MSE fournit une mesure quantitative pixel-à-pixel de la différence entre deux images, adaptée pour évaluer si l’approche GPIS produit des résultats visuellement équivalents au rendu maillé traditionnel.

La MSE se calcule comme suit :

$$\text{MSE} = \frac{1}{N} \sum (I_{\text{reference}} - I_{\text{gpis}})^2 \quad (11)$$

où  $N$  est le nombre total de pixels,  $I_{\text{reference}}$  représente les valeurs de pixels du tracé de rayon standard et  $I_{\text{gpis}}$  celles du renderer GPIS.

La MSE est pertinente car :

- Elle fournit une mesure objective et quantitative de similarité d'image
- Elle est largement utilisée en recherche en rendu pour comparer des images de référence
- Des valeurs de MSE plus faibles correspondent directement à des correspondances visuelles plus proches

#### 4.1.2 Performances de rendu

Nous comparerons les temps de rendu entre l'approche maillée traditionnelle et l'approche GPIS pour trois taux d'échantillonnage : 8, 32 et 128 échantillons par pixel. Cela permettra de quantifier le surcoût de performance lié à l'évaluation du processus gaussien et à l'échantillonnage corrélé. Sur la base du coût computationnel additionnel, on s'attend à un ralentissement de l'ordre de 510x par rapport au rendu maillé classique.

## 5 Discussion

Ce projet s'est avéré beaucoup plus difficile que prévu, présentant des obstacles tant sur le plan théorique que méthodologique et technique.

La lecture et la compréhension des articles ont été difficiles. La théorie des processus stochastiques nécessaire pour comprendre les GPIS dépasse certains aspects de mon bagage mathématique, m'imposant de revenir sur des notions fondamentales avant d'aborder le matériel avancé. En tant que première expérience de recherche formelle, j'ai d'abord eu des difficultés à analyser efficacement les articles; j'ai adopté une stratégie en deux passes : une lecture initiale pour saisir la structure globale, puis une seconde passe détaillée où je condense chaque paragraphe en une phrase en utilisant Obsidian pour organiser mes notes.

Le développement de la preuve de concept pour Mitsuba 3 a présenté des défis techniques importants. L'environnement de compilation a exigé une correspondance précise des versions (GCC, CMake, Ninja, Embree). De plus, la documentation limitée de l'API C++ de Mitsuba 3 a nécessité une rétro-ingénierie par inspection du code, et la maîtrise de la syntaxe DrJit a ajouté une couche de complexité supplémentaire.

Malgré ces difficultés, ce projet a été intellectuellement stimulant et professionnellement enrichissant. J'ai acquis une exposition précieuse aux développements en recherche en infographie, notamment l'intégration de techniques d'apprentissage automatique pour améliorer le photoréalisme et les applications pratiques. Le projet m'a aussi obligé à revisiter des concepts fondamentaux du tracé de rayon et des fonctions de diffusion, renforçant ma compréhension des sujets avancés au cœur de ce travail. Surtout, cette expérience a développé ma capacité à aborder des concepts théoriques complexes et à les appliquer à des bases de code de production existantes, compétence cruciale pour la recherche et le développement en infographie.

Bien que le projet ait été plus exigeant que prévu, j'anticipe des résultats significatifs de l'intégration des GPIS dans Mitsuba 3. Une part importante de théorie reste à explorer afin d'améliorer la rigueur de la revue de littérature, notamment le modèle de mémoire utilisé pour le transport de la lumière dans [1] et l'approche de convolution creuse plus

récente de Xu et al. [11]. Ces ajouts fourniront une base théorique plus complète pour le travail d'implémentation à venir.



## References

- [1] D. Seyb, E. d'Eon, B. Bitterli, and W. Jarosz, "From microfacets to participating media: A unified theory of light transport with stochastic geometry," *ACM Transactions on Graphics (Proceedings of SIGGRAPH)*, vol. 43, no. 4, Jul. 2024. DOI: [10/gt5nh9](https://doi.org/10.1145/gt5nh9).
- [2] J. T. Kajiya and T. L. Kay, "Rendering fur with three dimensional textures," in *Proceedings of the 16th Annual Conference on Computer Graphics and Interactive Techniques*, ser. SIGGRAPH '89, New York, NY, USA: ACM, 1989, pp. 271–280. DOI: [10.1145/74333.74361](https://doi.org/10.1145/74333.74361).
- [3] S. R. Marschner, H. W. Jensen, M. Cammarano, S. Worley, and P. Hanrahan, "Light scattering from human hair fibers," *ACM Transactions on Graphics*, vol. 22, no. 3, pp. 780–791, Jul. 2003, ISSN: 0730-0301. DOI: [10.1145/882262.882345](https://doi.org/10.1145/882262.882345). [Online]. Available: <https://doi.org/10.1145/882262.882345>.
- [4] M. J.-Y. Chiang, B. Bitterli, C. Tappan, and B. Burley, "A practical and controllable hair and fur model for production path tracing," *Computer Graphics Forum*, vol. 35, no. 2, pp. 275–283, 2016. DOI: <https://doi.org/10.1111/cgf.12830>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.12830>. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.12830>.
- [5] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. MIT Press, 2006, ISBN: 978-0-262-18253-9. [Online]. Available: <http://www.gaussianprocess.org/gpml/>.
- [6] W. Martens, Y. Poffet, P. Ramon, R. Fitch, and S. Sukkarieh, "Geometric priors for gaussian process implicit surfaces," *IEEE Robotics and Automation Letters*, vol. 2, no. 1, pp. 1–8, Nov. 2016. DOI: [10.1109/LRA.2016.2631260](https://doi.org/10.1109/LRA.2016.2631260).
- [7] O. Williams and A. Fitzgibbon, "Gaussian process implicit surfaces," in *Gaussian Processes in Practice*, Apr. 2007. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/gaussian-process-implicit-surfaces-2/>.
- [8] B. O. Community, *Blender - a 3d modelling and rendering package*, Stichting Blender Foundation, Amsterdam: Blender Foundation, 2018. [Online]. Available: <http://www.blender.org>.
- [9] Autodesk, Inc., *Arnold renderer*, version 7.3, <https://www.arnoldrenderer.com>, 2024.
- [10] B. Bitterli, *Tungsten renderer*, 2025. [Online]. Available: <https://github.com/tunabrain/tungsten/>.
- [11] K. Xu, B. Bitterli, E. d'Eon, and W. Jarosz, "Practical Gaussian process implicit surfaces with sparse convolutions," *ACM Transactions on Graphics (Proceedings of SIGGRAPH Asia)*, vol. 44, no. 6, Dec. 2025. DOI: [10.1145/3763329](https://doi.org/10.1145/3763329).
- [12] W. Jakob et al., *Mitsuba 3 renderer*, version 3.7.1, <https://mitsuba-renderer.org>, 2025.